

WEEK-5

REACT

React Lab Initialization

The objective of this hands-on lab was to understand the fundamentals of Single-Page Applications (SPAs) and React, including the Virtual DOM and the differences between SPAs and MPAs.

The practical requirements were to:

- Set up a local React development environment using Node.js and NPM.
- Initialize a new React application named myfirstreact using the create-react-app tool.
- Modify the default application code in App.js to render a specific heading: "**welcome to the first session of React**".
- Run the application and verify the output in a web browser on localhost:3000.

To complete the lab requirements using IntelliJ IDEA, the following steps were executed:

- **Project Creation:** Instead of running terminal commands manually, the built-in New Project wizard in IntelliJ IDEA was utilized. A new project named myfirstreact was created, explicitly configuring the React CLI dropdown to use create-react-app to match the lab's exact environment requirements.
- **Code Modification:** Located and opened the App.js file within the generated src directory. The default boilerplate code was completely removed and replaced with a clean functional component to display the requested heading

```
myfirstreact.iml  JS App.js x
1  import React from 'react';
2  import './App.css';
3
4  function App() { Show usages
5      return (
6          <div className="App">
7              <h1>welcome to the first session of React</h1>
8          </div>
9      );
10 }
11
12 export default App; Show usages
```

OUTPUT:



welcome to the first session of React

Student Management Portal Components

The objective of this hands-on lab was to understand React components, specifically focusing on the differences between functional and class components, the purpose of the render() function, and how to structure and invoke multiple components within a single application.

The practical requirements were to:

- Create a new React application named StudentApp.
- Create three distinct class components: Home, About, and Contact.
- Configure the Home component to display: **"Welcome to the Home page of Student Management Portal"**.
- Configure the About component to display: **"Welcome to the About page of the Student Management Portal"**.

- Configure the Contact component to display: **"Welcome to the Contact page of the Student Management Portal"**.
- Invoke (render) all three components simultaneously within the main application view.
- 🎬 **Project Initialization:** Created a new React project named StudentApp using IntelliJ's New Project wizard, utilizing the create-react-app CLI configuration for the initial setup.
- 📁 **Folder Structure Organization:** Created a dedicated Components directory inside the src folder to follow standard React project organization practices.
- 📄 **Component Creation:** Authored three separate JavaScript files (Home.js, About.js, and Contact.js) within the Components directory. Each file was implemented as a React Class Component, utilizing the render() method to return the specific required text wrapped in <h2> tags.
- 🎬 **Component Invocation:** Modified the main App.js file to import the newly created Home, About, and Contact components. Their respective JSX tags (<Home/>, <About/>, and <Contact/>) were added inside the main App component's return block so they would render together.
- 🎬 **Deployment & Verification:** Launched the development server via the integrated IntelliJ terminal using the npm start command. Verified that the browser successfully opened <http://localhost:3000> and displayed all three component messages on the page.

```
JS Home.js x JS About.js x JS Contact.js JS App.js
1 import React, { Component } from 'react';
2
3 class About extends Component { Show usages new *
4   render() { no usages new *
5     return (
6       <div>
7         <h2>Welcome to the About page of the Student Management Portal</h2>
8       </div>
9     );
10   }
11 }
12
13 export default About;| Show usages new *
```

```
JS Home.js JS About.js JS Contact.js x JS App.js
1 import React, { Component } from 'react';
2
3 class Contact extends Component { Show usages new *
4   render() { no usages new *
5     return (
6       <div>
7         <h2>Welcome to the Contact page of the Student Management Portal</h2>
8       </div>
9     );
10   }
11 }
12
13 export default Contact;| Show usages new *
```

```
JS Home.js x JS About.js JS Contact.js JS App.js
1 import React, { Component } from 'react';
2
3 class Home extends Component { Show usages new *
4   render() { no usages new *
5     return (
6       <div>
7         <h2>Welcome to the Home page of Student Management Portal</h2>
8       </div>
9     );
10   }
11 }
12
13 export default Home; Show usages new *
```

```
JS Home.js JS About.js JS Contact.js JS App.js x
1 import React from 'react';
2 import './App.css';
3 import Home from './Components/Home';
4 import About from './Components/About';
5 import Contact from './Components/Contact';
6
7 function App() { Show usages
8   return (
9     <div className="App">
10       <Home />
11       <About />
12       <Contact />
13     </div>
14   );
15 }
16
17 export default App; Show usages
```

OUTPUT:



Welcome to the Home page of Student Management Portal

Welcome to the About page of the Student Management Portal

Welcome to the Contact page of the Student Management Portal

Student Score Calculator App

The objective of this hands-on lab was to explore React functional components, understand how they differ from class components, learn how to pass data using properties (props), and apply external CSS styling to React elements.

The practical requirements were to:

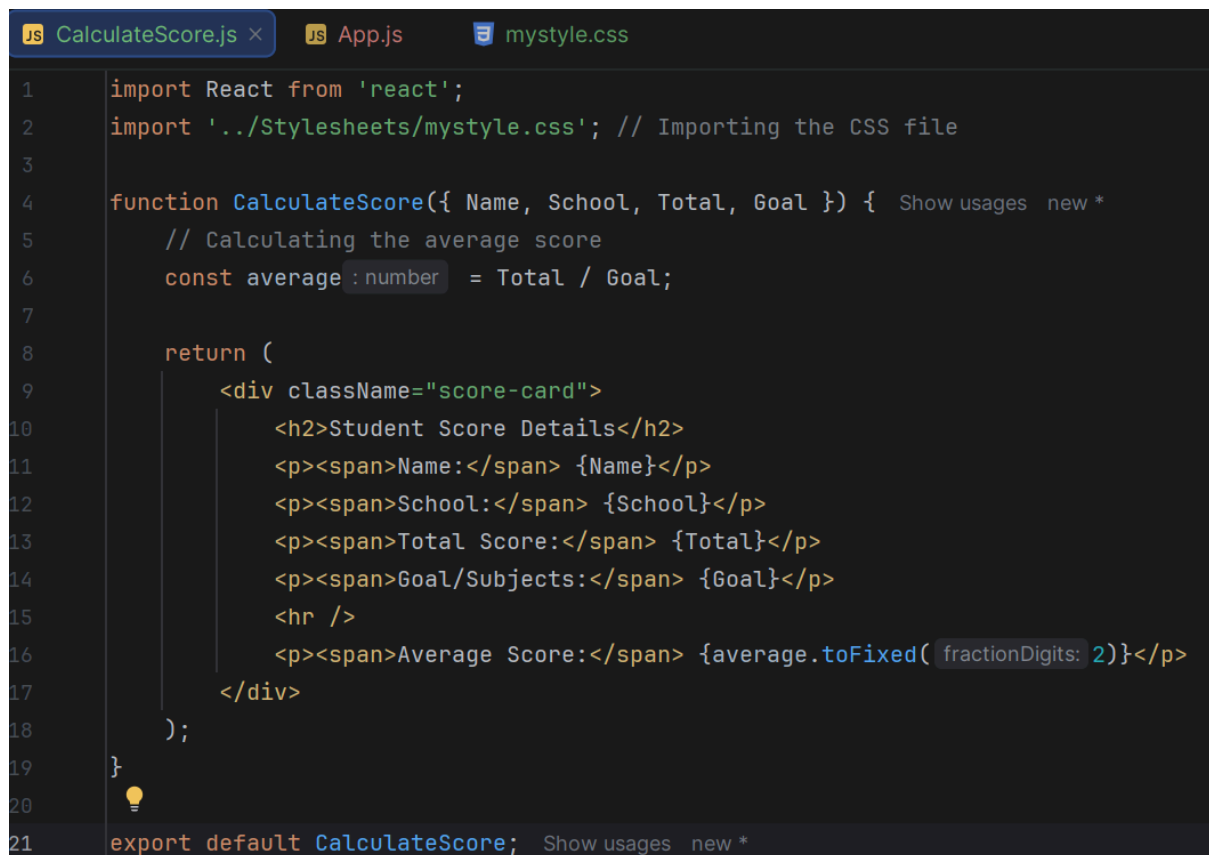
- Create a new React application named scorecalculatorapp.
- Create a functional component named CalculateScore that accepts four parameters (props): **Name**, **School**, **Total**, and **Goal**.
- Calculate the average score of the student based on the Total and Goal values provided.
- Create a custom stylesheet (mystyle.css) inside a Stylesheets folder to style the component visually.
- Invoke the CalculateScore component inside the main App.js file, passing in the required data, and run the application to view the output on localhost:3000

To complete the lab requirements using IntelliJ IDEA, the following steps were executed:

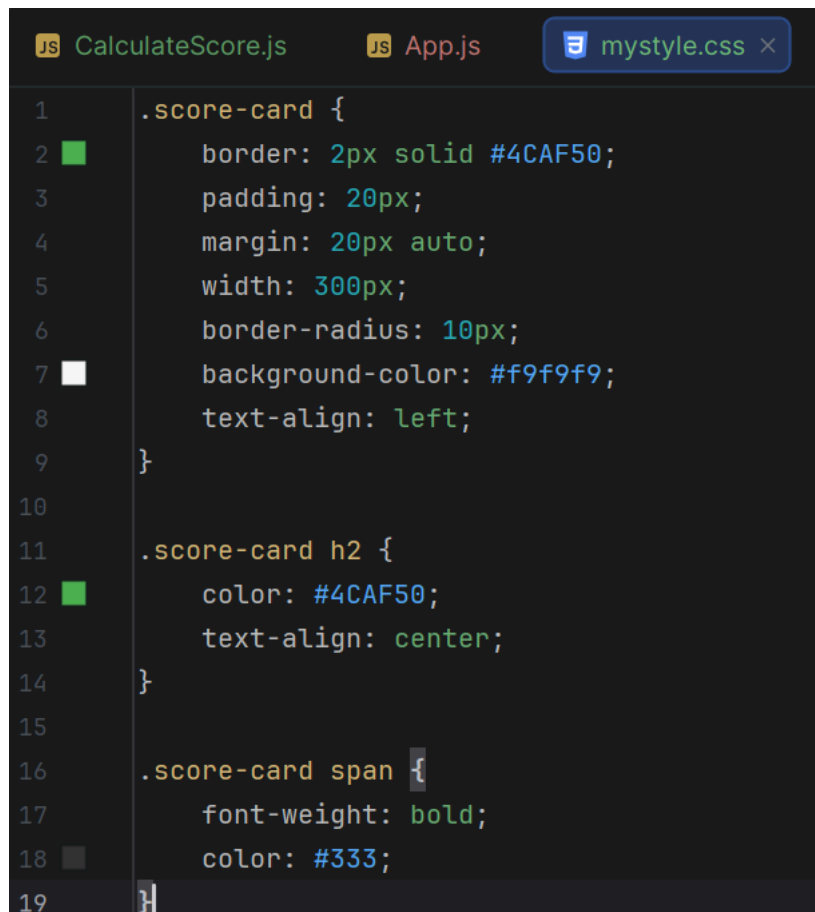
- **Project Initialization:** Created a new React project named scorecalculatorapp using the IntelliJ New Project wizard, specifically utilizing the create-react-app CLI layout to match the lab environment.
- **Folder & Style Setup:** Created two new directories within the src folder: Components and Stylesheets. Added a mystyle.css file to the Stylesheets folder, defining classes (like .score-card) to give

the output a clean, structured appearance with borders and centered text.

- **Functional Component Implementation:** Created CalculateScore.js in the Components folder. This file was written as a pure functional component that destructures { Name, School, Total, Goal } from its props. It calculates the average by dividing Total by Goal and returns a JSX structure displaying the student's details and their calculated average score, styled using the imported CSS.
- **Component Invocation:** Modified App.js to import the CalculateScore component. Invoked the component inside the main App return block and passed sample data (e.g., Name="John Doe", Total={450}) into the component as props.
- **Deployment & Verification:** Executed the npm start command in the IntelliJ integrated terminal. The development server successfully launched, displaying the styled score card and the correctly calculated average in the web browser at `



```
1  import React from 'react';
2  import '../Stylesheets/mystyle.css'; // Importing the CSS file
3
4  function CalculateScore({ Name, School, Total, Goal }) { Show usages new *
5    // Calculating the average score
6    const average : number = Total / Goal;
7
8    return (
9      <div className="score-card">
10        <h2>Student Score Details</h2>
11        <p><span>Name:</span> {Name}</p>
12        <p><span>School:</span> {School}</p>
13        <p><span>Total Score:</span> {Total}</p>
14        <p><span>Goal/Subjects:</span> {Goal}</p>
15        <hr />
16        <p><span>Average Score:</span> {average.toFixed( fractionDigits: 2)}</p>
17      </div>
18    );
19  }
20
21  export default CalculateScore; Show usages new *
```

A screenshot of a code editor with three tabs: 'CalculateScore.js', 'App.js', and 'mystyle.css'. The 'mystyle.css' tab is active, showing CSS code for a score card. The code is as follows:

```
1 .score-card {  
2     border: 2px solid #4CAF50;  
3     padding: 20px;  
4     margin: 20px auto;  
5     width: 300px;  
6     border-radius: 10px;  
7     background-color: #f9f9f9;  
8     text-align: left;  
9 }  
10  
11 .score-card h2 {  
12     color: #4CAF50;  
13     text-align: center;  
14 }  
15  
16 .score-card span {  
17     font-weight: bold;  
18     color: #333;  
19 }
```

React Component Lifecycle & Fetch API

The primary objective of this hands-on lab was to understand the React component lifecycle, specifically focusing on how to execute code at specific phases of a component's existence using lifecycle hook methods.

The practical requirements were to:

- Create a new React application named blogapp.
- Create a data model class named Post containing id, title, and body properties.
- Create a class-based component named Posts and initialize its state with an empty array of posts inside the constructor.
- Implement a custom loadPosts() method that uses the Fetch API to retrieve post data from a placeholder URL (jsonplaceholder.typicode.com/posts) and update the component's state.

- Implement the `componentDidMount()` lifecycle hook to trigger the `loadPosts()` fetch operation as soon as the component loads.
- Implement the `componentDidCatch()` lifecycle hook to catch and display any rendering errors as alert messages.
- Implement the `render()` method to display the fetched data, formatting titles as headings and post content as paragraphs.
- Invoke the `Posts` component inside the main `App.js` file

 **Project Initialization:** Created a new React project named `blogapp` using the `create-react-app` template within IntelliJ IDEA.

 **Data Model Creation:** Authored a standalone `Post.js` file that defines a standard JavaScript class with a constructor to structure the incoming `id`, `title`, and `body` data.

 **Component Implementation (Posts.js):**

- **State Setup:** Created a React class component, establishing `this.state = { posts: [] }` in the constructor.
- **Data Fetching:** Implemented `loadPosts()` to asynchronously fetch data from the provided API, map the resulting JSON array into instances of the `Post` class, and update the state using `this.setState()`.
- **Lifecycle Hooks:** Added `componentDidMount()` to automatically call `this.loadPosts()` exactly once when the component mounts. Added `componentDidCatch()` containing an `alert()` to capture and report unexpected errors gracefully.
- **Rendering UI:** Wrote the `render()` method to iterate through `this.state.posts` using the `.map()` function, rendering an `<h3>` heading for each title and a `<p>` tag for each body.

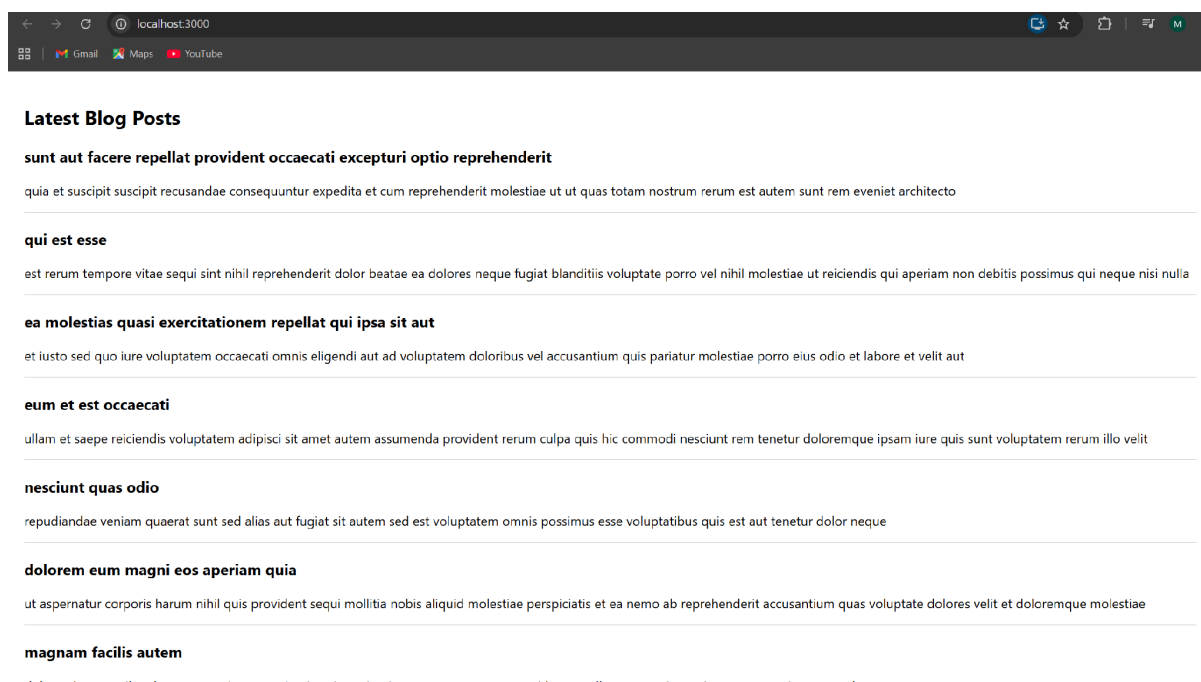
 **Integration & Deployment:** Modified `App.js` to import and render the `<Posts/>` component. Launched the application via the built-in terminal using `npm start` to verify that the API data was successfully fetched and displayed on `localhost:3000`.

```
JS Post.js x JS Posts.js JS App.js
1 class Post { Show usages new *
2   constructor(id, title, body) { Show usages new *
3     this.id = id;
4     this.title = title;
5     this.body = body;
6   }
7 }
8
9 export default Post;| Show usages new *
```

```
JS Post.js JS Posts.js x JS App.js
1 import React, { Component } from 'react';
2 import Post from './Post';
3
4 class Posts extends Component { Show usages new *
5   // Step 5: Initialize the component with a list of Posts in state
6   constructor(props) { no usages new *
7     super(props);
8     this.state = {
9       posts: []
10    };
11  }
12
13   // Step 6: Create loadPosts() using Fetch API
14   loadPosts(): void { Show usages new *
15     fetch(input: 'https://jsonplaceholder.typicode.com/posts') Promise<Response>
16       .then(response: Response => response.json()) Promise<any>
17       .then(data => {
18         // Mapping JSON data to our Post class
19         const postObjects = data.map(item => new Post(item.id, item.title, item.body));
20         this.setState( state: { posts: postObjects });
21       }) Promise<void>
22       .catch(error => {
23         console.error("Error fetching posts:", error);
24       });
25   }
```

```
JS Post.js JS Posts.js JS App.js x
1 import React from 'react';
2 import './App.css';
3 import Posts from './Posts'; // Importing the Posts component
4
5 function App() { Show usages
6   return (
7     <div className="App">
8       {/* Step 10: Add the Posts component to App component */}
9       <Posts />
10    </div>
11  );
12 }
13
14 export default App;| Show usages
```

OUTPUT:



React Router SPA for Trainers Portal

The objective of this hands-on lab was to understand the need for and benefits of React Router in building Single-Page Applications (SPAs). This included

identifying router components, defining routes, and passing parameters via the URL.

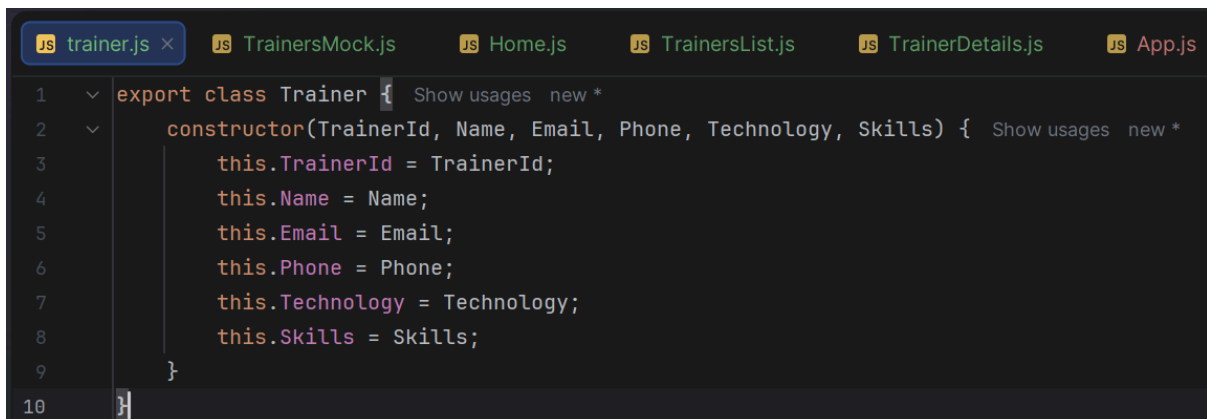
The practical requirements for the Cognizant Academy portal were to:

- Create a new React application named TrainersApp.
- Create a data model (Trainer class) containing specific properties: TrainerId, Name, Email, Phone, Technology, and Skills.
- Generate a mock data file (TrainersMock.js) populated with sample Trainer instances.
- Install and configure the react-router-dom library.
- Create three main UI components:
 - **Home:** A landing page with a welcome message.
 - **TrainersList:** A page displaying a list of trainers, where each name acts as a hyperlink.
 - **TrainerDetail:** A page that retrieves the trainer's ID from the URL parameters using the useParams hook to display their specific details.
- Configure the main App.js component to handle routing for the /, /trainers, and /trainers/:id paths, alongside a persistent navigation menu.

To complete the lab requirements using IntelliJ IDEA, the following steps were executed:

- **Project Initialization & Dependencies:** Created the React project via IntelliJ's project wizard using create-react-app. Opened the built-in terminal and executed npm install react-router-dom to add routing capabilities to the project.
- **Data Modeling:** Authored trainer.js to define the class structure and TrainersMock.js to instantiate an array of mock trainer data to simulate a backend response.
- **Component Development:**

- Created Home.js for the default landing view.
- Created TrainersList.js, which imports the mock data and uses the .map() function alongside the React Router <Link> component to generate dynamic URLs for each trainer (e.g., /trainers/1).
- Created TrainerDetails.js, utilizing the useParams hook to capture the URL parameter, match it against the mock data array, and render the specific data fields for the selected trainer.
- **Routing Configuration:** Restructured App.js by wrapping the application in a BrowserRouter. Added a <nav> bar with <Link> tags for top-level navigation, and defined <Routes> to map the specific URL paths (/ , /trainers, and /trainers/:id) to their respective components.
- **Deployment & Verification:** Ran the application using the npm start command. Verified in the browser (localhost:3000) that the navigation menu dynamically swapped the components without triggering a full page reload, successfully demonstrating SPA routing.



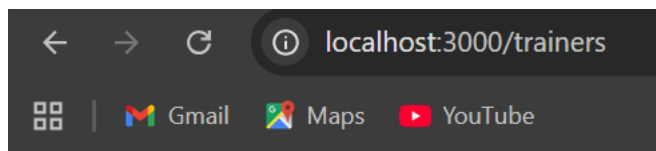
```
JS trainer.js x JS TrainersMock.js JS Home.js JS TrainersList.js JS TrainerDetails.js JS App.js
1  export class Trainer { Show usages new *
2      constructor(TrainerId, Name, Email, Phone, Technology, Skills) { Show usages new *
3          this.TrainerId = TrainerId;
4          this.Name = Name;
5          this.Email = Email;
6          this.Phone = Phone;
7          this.Technology = Technology;
8          this.Skills = Skills;
9      }
10
```

```
JS trainer.js × JS TrainersMock.js JS Home.js × JS TrainersList.js JS TrainerDetails.js JS App.js
1 import React from 'react';
2
3 export default function Home() { Show usages new *
4   return (
5     <div>
6       <h2>Welcome to Cognizant Academy Trainers Portal</h2>
7       <p>Select "Trainers List" from the menu to view our experts.</p>
8     </div>
9   );
10 }
```

```
JS trainer.js JS TrainersMock.js JS Home.js JS TrainersList.js × JS TrainerDetails.js JS App.js
1 import React from 'react';
2 import { Link } from 'react-router-dom';
3 import { trainersMock } from '../TrainersMock';
4
5 export default function TrainersList() { Show usages new *
6   return (
7     <div>
8       <h2>Trainers List</h2>
9       <ul>
10        {trainersMock.map(trainer: Trainer => (
11          <li key={trainer.TrainerId} style={{ margin: '10px 0' }}>
12            /* Creates a dynamic URL using the TrainerId */
13            <Link to={`/${trainer.TrainerId}`}>
14              {trainer.Name}
15            </Link>
16          </li>
17        ))}
18      </ul>
19    </div>
20  );
21 }
```

```
JS trainer.js JS TrainersMock.js JS Home.js JS TrainersList.js JS TrainerDetails.js x JS App.js
1 > import ...
4
5 export default function TrainerDetail() { Show usages
6   // Extract the ID parameter from the URL
7   const { id : string } = useParams();
8
9   // Find the trainer whose ID matches the URL parameter
10  const trainer : Trainer = trainersMock.find(t : Trainer => t.TrainerId.toString() === id);
11
12  if (!trainer) {
13    return <div><h2>Trainer not found!</h2></div>;
14  }
15
16  return (
17    <div style={{ border: '1px solid #ccc', padding: '20px', width: '300px' }}>
18      <h2>Trainer Details</h2>
19      <p><strong>T-ID:</strong> {trainer.TrainerId}</p>
20      <p><strong>Name:</strong> {trainer.Name}</p>
21      <p><strong>Email:</strong> {trainer.Email}</p>
22      <p><strong>Phone:</strong> {trainer.Phone}</p>
23      <p><strong>Stream:</strong> {trainer.Technology}</p>
24      <p><strong>Skills:</strong> {trainer.Skills}</p>
25
26      <br />
27      <Link to="/trainers">← Back to Trainers List</Link>
28    </div>
29  );
30
```

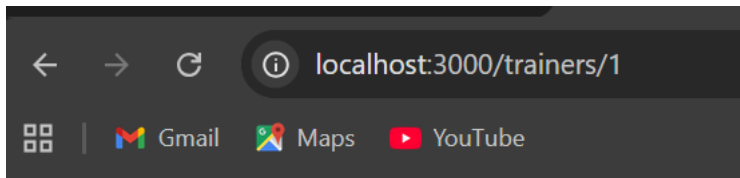
OUTPUT:



[Home](#) [Trainers List](#)

Trainers List

- [John Doe](#)
- [Jane Smith](#)
- [Sam Wilson](#)



[Home](#) [Trainers List](#)

Trainer Details

T-ID: 1

Name: John Doe

Email: john.doe@cognizant.com

Phone: 9876543210

Stream: React

Skills: SPA, Hooks, Redux

[← Back to Trainers List](#)

WEEK-5

Cricket App (ES6 Features Integration)

The objective of this hands-on lab was to understand and apply modern ES6 JavaScript features within a React environment. This included using arrow functions, the `map()` method, array destructuring, the spread operator, and managing conditional rendering blocks.

The practical requirements were to:

- Create a new React application named `cricketapp`.
- Build a `ListofPlayers` component that declares an array of 11 players with names and scores, renders the complete list using the ES6 `map` feature, and uses an ES6 arrow function to filter and display players with scores below 70.
- Build an `IndianPlayers` component that uses ES6 array destructuring to separate and display odd-position and even-position team players, and uses the ES6 merge feature (spread operator) to combine separate T20 and Ranji Trophy player arrays into a single list.
- Integrate both components into the main home page (`App.js`), using a simple if-else condition controlled by a flag variable to display `ListofPlayers` when the flag is true, and `IndianPlayers` when the flag is false.

 **Project Setup:** Created the React project named `cricketapp` using the built-in IntelliJ project wizard, selecting the standard create-react-app CLI template configurations.

 **Component Development:** Created a dedicated Components directory within `src` and added two functional components:

- **ListofPlayers.js:** Declared the 11-player data structure. Used `.map()` to generate list items for all players. Implemented a `.filter()` method with an ES6 arrow function (`player => player.score < 70`) to create a secondary filtered view.
- **IndianPlayers.js:** Implemented ES6 array destructuring (`const [player1, player2, player3, player4] = topPlayers`) to extract individual index

positions for the odd and even display. Used the spread operator ([...T20players, ...RanjiTrophyPlayers]) to merge separate player pools into a unified array.

🔧 **Conditional Page Routing:** Updated the main App.js file by introducing a local boolean flag variable. Used a standard JSX ternary expression (flag ? <ListofPlayers/> : <IndianPlayers/>) to execute the required conditional rendering layout.

🔧 **Execution:** Launched the development environment from the integrated IntelliJ terminal using the npm start command and verified the dynamic UI output changes on localhost:3000 by toggling the flag between true and false states.

```
1  import React from 'react';
2
3  function IndianPlayers() { Show usages new *
4      const topPlayers : string[] = ["Rohit", "Virat", "Rahul", "Gill"];
5
6      // Destructuring array elements into distinct variables
7      const [player1 : string , player2 : string , player3 : string , player4 : string ] = topPlayers;
8
9      // Declaring two separate arrays
10     const T20players : string[] = ["Surya", "Hardik", "Bumrah"];
11     const RanjiTrophyPlayers : string[] = ["Sarfaraz", "Iyer", "Jaiswal"];
12
13     // Merging the two arrays using the ES6 Spread Operator (...)
14     const mergedPlayers : string[] = [...T20players, ...RanjiTrophyPlayers];
15
16     return (
17         <div>
18             <h3>Team Selection (ES6 Destructuring)</h3>
19             <p><strong>Odd Position Players (1st & 3rd):</strong> {player1}, {player3}</p>
20             <p><strong>Even Position Players (2nd & 4th):</strong> {player2}, {player4}</p>
21
22             <h3>Merged Squad List (ES6 Spread Operator)</h3>
23             <ul>
24                 {mergedPlayers.map((player : string , index : number ) => (
25                     <li key={index}>{player}</li>
26                 ))}
27             </ul>
28         </div>
29     );
30 }
31
32 export default IndianPlayers; Show usages new *
```

```
JS ListofPlayers.js x JS IndianPlayers.js JS App.js
3 function ListofPlayers() { Show usages new *
5     const players : {(name: string, score: number)...} = [
6         { name: "Virat Kohli", score: 85 },
7         { name: "Rohit Sharma", score: 92 },
8         { name: "KL Rahul", score: 45 },
9         { name: "Shubman Gill", score: 68 },
10        { name: "Shreyas Iyer", score: 74 },
11        { name: "Rishabh Pant", score: 55 },
12        { name: "Hardik Pandya", score: 62 },
13        { name: "Ravindra Jadeja", score: 38 },
14        { name: "R. Ashwin", score: 25 },
15        { name: "Jasprit Bumrah", score: 12 },
16        { name: "Mohammed Shami", score: 8 }
17    ];
18
19    // Filtering players with scores below 70 using an ES6 arrow function
20    const lowScoringPlayers : {(...)}[] = players.filter(player : {name: string, score: number} => player.score < 70)
21
22    return (
23        <div>
24            <h3>All Players List (ES6 Map)</h3>
25            <ul>
26                {players.map((player : {name: string, score: number} , index : number ) => (
27                    <li key={index}>{player.name} - Score: {player.score}</li>
28                ))}
29            </ul>
30
31            <h3>Players with Scores Below 70 (ES6 Filter & Arrow Function)</h3>
32            <ul>
33                {lowScoringPlayers.map((player : {name: string, score: number} , index : number ) => (
34                    <li key={index}>{player.name} - Score: {player.score}</li>
35                ))}
36            </ul>
```

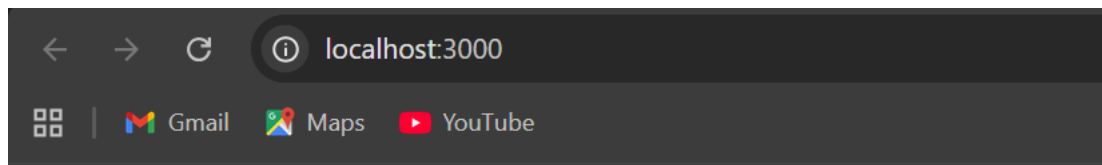
```
import React from 'react';
import './App.css';
import ListofPlayers from './Components/ListofPlayers';
import IndianPlayers from './Components/IndianPlayers';

function App() {  Show usages
  // Toggle this flag variable between true and false to change the rendering output
  const flag: boolean = true;

  return (
    <div className="App" style={{ padding: '30px', textAlign: 'left' }}>
      <h2>Cricket Application Dashboard</h2>
      <p>Current Flag Status: <strong>{flag.toString()}</strong></p>
      <hr />

      {/* Simple conditional rendering based on the flag variable */}
      {flag ? <ListofPlayers /> : <IndianPlayers />}
    </div>
  );
}
```

OUTPUT:



Cricket Application Dashboard

Current Flag Status: **true**

All Players List (ES6 Map)

- Virat Kohli — Score: 85
- Rohit Sharma — Score: 92
- KL Rahul — Score: 45
- Shubman Gill — Score: 68
- Shreyas Iyer — Score: 74
- Rishabh Pant — Score: 55
- Hardik Pandya — Score: 62
- Ravindra Jadeja — Score: 38
- R. Ashwin — Score: 25
- Jasprit Bumrah — Score: 12
- Mohammed Shami — Score: 8

Players with Scores Below 70 (ES6 Filter & Arrow Function)

- KL Rahul — Score: 45
- Shubman Gill — Score: 68
- Rishabh Pant — Score: 55
- Hardik Pandya — Score: 62
- Ravindra Jadeja — Score: 38
- R. Ashwin — Score: 25
- Jasprit Bumrah — Score: 12
- Mohammed Shami — Score: 8

Office Space Rental App (JSX & Inline Styling)

The objective of this hands-on lab was to master React JSX fundamentals, including creating elements, managing JSX attributes, embedding JavaScript expressions, and implementing dynamic inline CSS styling.

The practical requirements were to:

- Create a new React application named `officespacerentalapp`.
- Store and render the main directory heading inside a JSX element variable.
- Pass image source URLs dynamically using standard JSX image attributes.
- Create a standalone JavaScript object representing an office property to display its details (Name, Rent, and Address).
- Create an array containing a list of multiple office objects and loop through them to generate a repeating layout structure.
- Apply conditional inline CSS to the rent display: format the text color in **Red** if the rent is below 60,000, and in **Green** if the rent is 60,000 or above.

 **Project Initialization:** Spawned a new React application directory named `officespacerentalapp` utilizing IntelliJ's New Project setup pane configured with the standard create-react-app CLI layout template.

 **Component Architecture:** Established a dedicated Components directory under the `src` root folder and built a custom `OfficeSpace.js` functional component file to isolate the app UI logic.

Data Modeling & JSX Setup:

- Declared a constant variable holding an `<h1>` element block for the page header.
- Structured a `featuredOffice` block as a single entity, and an `officeList` array populated with multiple property mock objects containing distinct data for names, rent pricing integers, locations, and image URLs.

Dynamic Map & Conditional Styling Insertion:

- Wrote the layout markup to output the individual featured card, loading the image asset via the standard JSX `src={}` attribute layout.

- Utilized an ES6 .map() function expression block to loop through the officeList array and uniformly duplicate the template design for all entries.
- Embedded a conditional JavaScript ternary operator directly into the inline style wrapper (style={{ color: office.rent < 60000 ? 'red' : 'green' }}) to toggle the visual color rendering engine dynamically based on pricing boundaries.

 **Mounting & Deployment:** Rewrote App.js to clean out standard boilerplate configurations, imported OfficeSpace, and nested the <OfficeSpace/> element structure directly into the target execution markup tree. Ran npm start within the terminal window to successfully verify the operational behavior at localhost:3000

```
import React from 'react';
```

```
function OfficeSpace() {
```

```
  const pageHeading = <h1>Office Space Rental Directory</h1>;
```

```
  const featuredOffice = {
```

```
    name: "Alpha Premium Headquarters",
```

```
    rent: 75000,
```

```
    address: "123 Tech Park Avenue, Suite 400",
```

```
    image:
```

```
    "https://images.unsplash.com/photo-1497366216548-37526070297c?auto=format&fit=crop&w=400&h=250&q=80"
```

```
  };
```

```
  const officeList = [
```

```
    {
```

```
      name: "Downtown Open Coworking",
```

```
      rent: 45000,
```

```
      address: "56 Corporate Boulevard",
```

```
    image:
      "https://images.unsplash.com/photo-1539635278303-d4002c07eae3?auto=format&fit=crop&w=400&h=250&q=80"
    },
    {
      name: "Skyline Executive Hub",
      rent: 85000,
      address: "88 Tower Road, Floor 22",
      image:
        "https://images.unsplash.com/photo-1497215728101-856f4ea42174?auto=format&fit=crop&w=400&h=250&q=80"
    },
    {
      name: "Suburban Innovation Lab",
      rent: 58000,
      address: "102 Technology Way",
      image:
        "https://images.unsplash.com/photo-1524758631624-e2822e304c36?auto=format&fit=crop&w=400&h=250&q=80"
    }
  ];
```

```
return (
  <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
    { /* Rendering the heading element */ }
    {pageHeading}

    { /* Rendering a single object */ }
```



```
<div style={{ border: '2px solid #333', padding: '15px', marginBottom: '30px',
borderRadius: '8px', maxWidth: '400px' }}>
```

```
<h2>Featured Property</h2>
```

```
<img src={featuredOffice.image} alt={featuredOffice.name} style={{ width:
'100%', borderRadius: '4px' }} />
```

```
<h3>{featuredOffice.name}</h3>
```

```
<p><strong>Address:</strong> {featuredOffice.address}</p>
```

```
<p>
```

```
<strong>Monthly Rent: </strong>
```

```
{/* Conditional inline CSS styling */}
```

```
<span style={{ color: featuredOffice.rent < 60000 ? 'red' : 'green',
fontWeight: 'bold' }}>
```

```
₹{featuredOffice.rent}
```

```
</span>
```

```
</p>
```

```
</div>
```

```
{/* Rendering a list of objects using map() */}
```

```
<h2>All Available Spaces</h2>
```

```
<div style={{ display: 'flex', gap: '20px', flexWrap: 'wrap' }}>
```

```
{officeList.map((office, index) => (
```

```
<div key={index} style={{ border: '1px solid #ccc', padding: '15px',
borderRadius: '8px', width: '280px' }}>
```

```
<img src={office.image} alt={office.name} style={{ width: '100%', height:
'150px', objectFit: 'cover', borderRadius: '4px' }} />
```

```
<h3>{office.name}</h3>
```

```
<p><strong>Address:</strong> {office.address}</p>
```

```
<p>
```

```

    <strong>Monthly Rent: </strong>

    /* Conditional inline CSS styling */
    <span style={{ color: office.rent < 60000 ? 'red' : 'green', fontWeight:
'bold' }}>
        ₹{office.rent}
    </span>

</p>

</div>

)}}

</div>

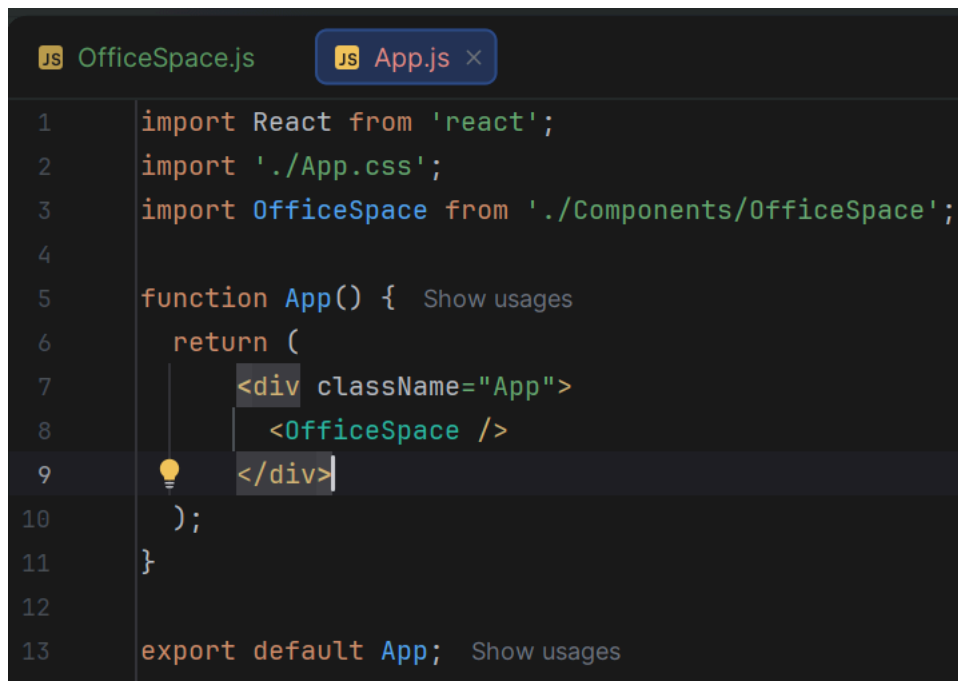
</div>

);

}

export default OfficeSpace;

```



```

JS OfficeSpace.js  JS App.js x
1  import React from 'react';
2  import './App.css';
3  import OfficeSpace from './Components/OfficeSpace';
4
5  function App() { Show usages
6      return (
7          <div className="App">
8              <OfficeSpace />
9              </div>
10         );
11     }
12
13     export default App; Show usages

```

OUTPUT:



Office Space Rental Directory

Featured Property



Alpha Premium Headquarters

Address: 123 Tech Park Avenue, Suite 400

Monthly Rent: ₹75000

All Available Spaces



Downtown Open Coworking

Address: 56 Corporate Boulevard

Monthly Rent: ₹45000



Skyline Executive Hub

Address: 88 Tower Road, Floor 22

Monthly Rent: ₹85000



Suburban Innovation Lab

Address: 102 Technology Way

Monthly Rent: ₹58000

React Events and Forms Application

The objective of this hands-on lab was to understand React event systems, including event handlers, naming conventions, synthetic events, and managing execution context using the `this` keyword.

The practical requirements were to:

- Create a new React application named `eventexamplesapp`.
- Build an interactive counter with an "Increment" button that simultaneously updates the numerical state and invokes a secondary method to display a static "Hello" alert message.
- Build a "Decrement" button to decrease the counter value.
- Build a "Say Welcome" button that passes a custom text string ('welcome') as an argument to an alert function.
- Build a button that demonstrates a React Synthetic Event by intercepting the click and displaying an alert message reading "I was clicked".
- Build a `CurrencyConvertor` component that handles form submission through a `handleSubmit` event, dynamically converting Indian Rupees (INR) to Euros (EUR) and rendering the result.
- **Project Initialization:** Created a new React project directory named `eventexamplesapp` using the `create-react-app` architecture within the IntelliJ project wizard.
- **Component Structure:** Established a `Components` directory inside the `src` folder to cleanly isolate feature logic.
- **Class Component Implementation (`EventExamples.js`):** Implemented a class-based component to explicitly demonstrate the `this` keyword. Bound event handlers in the constructor to maintain correct execution context. Configured the increment logic to chain two internal functions, and wrapped the argument passing requirement in an inline arrow function definition.
- **Form & Math Processing (`CurrencyConvertor.js`):** Built a form structure that reads user input into the local state. Used the synthetic event's `e.preventDefault()` method inside `handleSubmit` to suppress the default browser page reload, isolating the conversion math safely within the UI tree.
- **Assembly & Mounting:** Cleaned out the boilerplate code in `App.js`, imported both functional files, and mounted them into a single coherent dashboard. Launched the development server using `npm start` in the terminal to verify all interactive elements on `localhost:3000`.

Project Source Code

File Name: src/Components/EventExamples.js

JavaScript

```
import React, { Component } from 'react';

class EventExamples extends Component {
  constructor(props) {
    super(props);
    this.state = { counter: 0 };

    // Explicit binding to demonstrate the 'this' keyword requirement
    this.handleIncrement = this.handleIncrement.bind(this);
    this.handleDecrement = this.handleDecrement.bind(this);
    this.handleSyntheticEvent = this.handleSyntheticEvent.bind(this);
  }

  // Static message method invoked alongside counter incrementing
  sayHello() {
    alert("Hello! Welcome to the React Event Handling session.");
  }

  handleIncrement() {
    // Method part A: Update state
    this.setState(prevState => ({ counter: prevState.counter + 1 }));
    // Method part B: Invoke secondary method
    this.sayHello();
  }

  handleDecrement() {
    this.setState(prevState => ({ counter: prevState.counter - 1 }));
  }

  // Method handling custom arguments
  sayWelcome(message) {
    alert(`Argument passed: ${message}`);
  }

  // Method handling React Synthetic Event properties
  handleSyntheticEvent(e) {
    alert("I was clicked");
    console.log("Synthetic Event Type:", e.type);
  }

  render() {
    return (
      <div style={{ border: '1px solid #ccc', padding: '20px', marginBottom: '20px', borderRadius: '8px' }}>
        <h3>1. Counter (Multiple Methods & 'this' Binding)</h3>
        <p>Counter: <strong>{this.state.counter}</strong></p>
        <button onClick={this.handleIncrement}>Increment</button>
        <button onClick={this.handleDecrement} style={{ marginLeft: '10px' }}>Decrement</button>

        <h3>2. Function with Arguments</h3>
        <button onClick={() => this.sayWelcome('welcome')}>Say Welcome</button>

        <h3>3. Synthetic Event</h3>
        <button onClick={this.handleSyntheticEvent}>Trigger Event</button>
      </div>
    );
  }
}
```

```

    );
  }
}

export default EventExamples;

```

File Name: src/Components/CurrencyConvertor.js

JavaScript

```

import React, { Component } from 'react';
class CurrencyConvertor extends Component {
  constructor(props) {
    super(props);
    this.state = {
      inr: '',
      euro: 0
    };
    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  handleChange(e) {
    this.setState({ inr: e.target.value });
  }

  handleSubmit(e) {
    e.preventDefault();
    const inputRupees = parseFloat(this.state.inr);
    if (!isNaN(inputRupees) && inputRupees >= 0) {
      const calculatedEuro = inputRupees / 90;
      this.setState({ euro: calculatedEuro.toFixed(2) });
    } else {
      alert("Please enter a valid amount in INR.");
    }
  }

  render() {
    return (
      <div style={{ border: '1px solid #ccc', padding: '20px', borderRadius:
'8px' }}>
        <h3>4. Currency Converter (INR to Euro)</h3>
        <form onSubmit={this.handleSubmit}>
          <div style={{ marginBottom: '10px' }}>
            <label>Enter Rupees (INR): </label>
            <input
              type="number"
              value={this.state.inr}
              onChange={this.handleChange}
              placeholder="e.g. 9000"
              style={{ padding: '5px', marginLeft: '10px' }}
            />
          </div>
          <button type="submit" style={{ padding: '6px 12px', cursor:
'pointer' }}>Convert</button>
        </form>
        <p style={{ marginTop: '15px' }}>
          Resulting Value: <strong>€ {this.state.euro}</strong>
        </p>
      </div>
    );
  }
}

```

```
}  
}  
export default CurrencyConvertor;
```

OUTPUT:

React Interactive Event Dashboard

1. Counter (Multiple Methods & 'this' Binding)

Counter: **2**

Increment

Decrement

2. Function with Arguments

Say Welcome

3. Synthetic Event

Trigger Event

4. Currency Converter (INR to Euro)

Enter Rupees (INR):

89

Convert

Resulting Value: **€ 0.99**

Ticket Booking Application

The primary objective of this hands-on lab was to master conditional rendering strategies in React. This involved using element variables to dynamically store blocks of JSX, implementing conditional statements, and learning how to prevent a component from rendering anything to the DOM.

The practical requirements for the system were to:

- Create a new React application named ticketbookingapp.

- Build a conditional views workflow where a guest user can only browse an informational list of available flights without the ability to book tickets.
- Allow a logged-in user to view an authorized booking dashboard containing functional "Book Ticket" buttons for each flight.
- Implement dynamic Login and Logout actions that change the app layout based on a boolean tracking state.
- Demonstrate core React rendering paradigms: element variables, short-circuit operators (&&), and blocking component layouts by returning null.

What Was Done (Implementation & Execution) To complete the lab requirements using IntelliJ IDEA, the following steps were executed:

- **Project Initialization:** Spawned a new React application workspace named ticketbookingapp using the create-react-app CLI template configurations.
- **Component Architecture:** Established a dedicated Components directory under the src root folder to modularize feature execution.
- **Preventing Render Layout (ExclusiveOffers.js):** Developed a functional component that accepts an authorization prop. Programmed an explicit check (if (!isLoggedIn) return null;) to verify that a component can completely block its own UI output when conditions are unmet.
- **Element Variable Architecture (FlightBooking.js):** Built a class component featuring local state properties. Declared empty script scope variables (authButton, viewGreeting, tableActionHeader) inside the render() block and loaded them with specific JSX code fragments inside an if-else block depending on the user's login status.
- **Dynamic Grid Mapping:** Configured an iterative list array using .map() to output custom rows for each flight. Embedded short-circuit execution steps directly in the table row structure to toggle the active booking column based on the login state.
- **Integration Testing:** Connected the complete page structure into App.js. Ran terminal deployment routines using npm start to confirm that headers, columns, action buttons, and discount blocks dynamically updated in the browser at localhost:3000.

Project Source Code

File Name: src/Components/ExclusiveOffers.js

JavaScript

```
import React from 'react';
```

```
function ExclusiveOffers({ isLoggedIn }) {
```



```
// Preventing the component from rendering entirely if user is a guest
if (!isLoggedIn) {
  return null;
}

return (
  <div style={{ backgroundColor: '#fff3cd', padding: '15px', marginTop: '20px',
borderRadius: '5px', border: '1px solid #ffeeba', color: '#856404' }}>
    <strong>🔥 Member Exclusive Deal:</strong> Use code
<strong>FLY2026</strong> at checkout to save 10% on your selected flight
ticket!
  </div>
);
}
```

export default ExclusiveOffers;

File Name: src/Components/FlightBooking.js

JavaScript

import React, { Component } from 'react';

import ExclusiveOffers from './ExclusiveOffers';

```
class FlightBooking extends Component {
  constructor(props) {
    super(props);
    this.state = {
      isLoggedIn: false
    };
    this.handleLogin = this.handleLogin.bind(this);
    this.handleLogout = this.handleLogout.bind(this);
  }
```

```
  handleLogin() {
    this.setState({ isLoggedIn: true });
  }
```

```
  handleLogout() {
    this.setState({ isLoggedIn: false });
  }
```

```
  render() {
```

```

const { isLoggedIn } = this.state;

// Declaring Element Variables to demonstrate conditional markup storage
let authButton;
let viewGreeting;
let tableActionHeader;

if (isLoggedIn) {
  // Element variables populated for logged-in user page view
  authButton = <button onClick={this.handleLogout} style={{
backgroundColor: '#dc3545', color: 'white', padding: '8px 16px', border: 'none',
borderRadius: '4px', cursor: 'pointer' }}>Logout</button>;
  viewGreeting = <h2 style={{ color: '#0056b3' }}>User Dashboard: Registered
Ticket Booking Mode</h2>;
  tableActionHeader = <th style={{ padding: '12px', borderBottom: '2px solid
#ddd' }}>Purchase Option</th>;
} else {
  // Element variables populated for standard anonymous guest browsing
view
  authButton = <button onClick={this.handleLogin} style={{ backgroundColor:
'#28a745', color: 'white', padding: '8px 16px', border: 'none', borderRadius:
'4px', cursor: 'pointer' }}>Login</button>;
  viewGreeting = <h2 style={{ color: '#6c757d' }}>Guest Portal: Read-Only
Flight Directory Mode</h2>;
  tableActionHeader = null; // Prevents this specific column title node from
rendering
}

// Static Flight Array
const flights = [
  { id: 201, flightNo: "AI-102", route: "New Delhi (DEL) to Mumbai (BOM)",
price: "₹6,400" },
  { id: 202, flightNo: "6E-432", route: "Hyderabad (HYD) to Bengaluru (BLR)",
price: "₹3,900" },
  { id: 203, flightNo: "UK-811", route: "Kolkata (CCU) to Chennai (MAA)",
price: "₹7,200" }
];

return (

```

```

<div style={{ padding: '30px', fontFamily: 'Arial, sans-serif', maxWidth:
'800px', margin: '0 auto' }}>
  <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', borderBottom: '3px solid #343a40', paddingBottom: '15px' }}>
    <h1 style={{ margin: 0 }}>SkyRoute Airways</h1>
    {/* Rendering the button element variable */}
    {authButton}
  </div>

  <div style={{ marginTop: '20px' }}>
    {/* Rendering the greeting message element variable */}
    {viewGreeting}
  </div>

  <table style={{ width: '100%', marginTop: '20px', borderCollapse: 'collapse',
textAlign: 'left' }}>
    <thead>
      <tr style={{ backgroundColor: '#f8f9fa' }}>
        <th style={{ padding: '12px', borderBottom: '2px solid #ddd' }}>Flight
Number</th>
        <th style={{ padding: '12px', borderBottom: '2px solid #ddd' }}>Route
Details</th>
        <th style={{ padding: '12px', borderBottom: '2px solid #ddd'
}}>Standard Price</th>
        {/* Rendering table header column via element variable */}
        {tableActionHeader}
      </tr>
    </thead>
    <tbody>
      {flights.map(flight => (
        <tr key={flight.id} style={{ borderBottom: '1px solid #eee' }}>
          <td style={{ padding: '12px' }}>{flight.flightNo}</td>
          <td style={{ padding: '12px' }}>{flight.route}</td>
          <td style={{ padding: '12px' }}>{flight.price}</td>

          {/* Inline Conditional Short-Circuit Execution for Booking Buttons */}
          {isLoggedIn && (
            <td style={{ padding: '12px' }}>
              <button

```

```

        onClick={() => alert(`Success! Seat reserved on flight
${flight.flightNo}.`)}
        style={{ backgroundColor: '#007bff', color: 'white', border: 'none',
padding: '6px 12px', borderRadius: '4px', cursor: 'pointer' }}
      >
        Book Ticket
      </button>
    </td>
  )}
</tr>
)}}
</tbody>
</table>

```

```

    { /* Passing state property down to show dynamic component rejection
pattern */}
    <ExclusiveOffers isLoggedIn={isLoggedIn} />
  </div>
);
}
}

```

export default FlightBooking;

File Name: src/App.js

JavaScript

import React from 'react';

import './App.css';

import FlightBooking from './Components/FlightBooking';

```

function App() {
  return (
    <div className="App">
      <FlightBooking />
    </div>
  );
}

```

export default App;

OUTPUT:

SkyRoute Airways

Login

Guest Portal: Read-Only Flight Directory Mode

Flight Number	Route Details	Standard Price
AI-102	New Delhi (DEL) to Mumbai (BOM)	₹6,400
6E-432	Hyderabad (HYD) to Bengaluru (BLR)	₹3,900
UK-811	Kolkata (CCU) to Chennai (MAA)	₹7,200

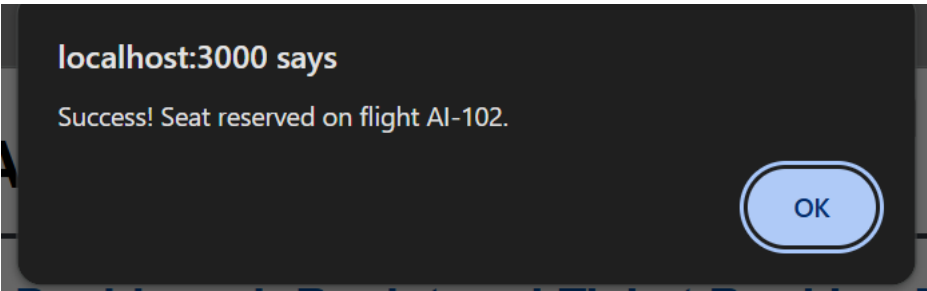
SkyRoute Airways

Logout

User Dashboard: Registered Ticket Booking Mode

Flight Number	Route Details	Standard Price	Purchase Option
AI-102	New Delhi (DEL) to Mumbai (BOM)	₹6,400	<button>Book Ticket</button>
6E-432	Hyderabad (HYD) to Bengaluru (BLR)	₹3,900	<button>Book Ticket</button>
UK-811	Kolkata (CCU) to Chennai (MAA)	₹7,200	<button>Book Ticket</button>

🔥 **Member Exclusive Deal:** Use code **FLY2026** at checkout to save 10% on your selected flight ticket!



Blogger Application

The objective of this hands-on lab was to understand lists, keys, and advanced conditional rendering methods in React.

The requirements for this project were to:

- Create a React application named bloggerapp.

- Build three distinct components: Book Details, Blog Details, and Course Details.
- Implement as many conditional rendering techniques as possible (such as if-else logic, element variables, ternary operators, short-circuit evaluation, and returning null).
- Demonstrate rendering multiple components inside a main view.
- Use the JavaScript map() function to render lists, explain the purpose of unique key props, and show how to safely extract child components with keys.

What Was Done (Implementation & Execution) To complete the lab requirements using IntelliJ IDEA 2026.1.3, the following tasks were carried out:

- **Project Workspace Setup:** Generated the base app repository labeled bloggerapp using the create-react-app framework template configurations.
- **Element Variable Switchboard (BloggerDashboard.js):** Built a main dashboard with tab buttons to manage multiple child views. It uses an **element variable** (displayMarkupContent) populated by an if-else statement to switch between the three primary feature components based on state.
- **Component Extraction & Keys (BookDetails.js):** Implemented an array of book data looped over by a **React Map (map())** statement. Extracted the list design into a separate child component (BookItem) and assigned the unique key prop to the custom element in the parent loop to keep the DOM stable.
- **Short-Circuit Evaluation (BlogDetails.js):** Built a list layout displaying blog posts. Used an inline **short-circuit operator (&&)** to look at each item's property and dynamically attach a "Featured Star" badge only if the value is true.
- **Ternary Rendering & Component Hiding (CourseDetails.js):** Created a conditional switch that alternates layout shapes (List view vs. Grid view) instantly using a **ternary operator (? :)**. Also included a system banner component (MaintenanceBanner) that completely **prevents rendering by returning null** when a toggle condition is met.
- **System Assembly:** Connected the main dashboard inside App.js and confirmed smooth performance across all conditional paths in the browser.

Project Source Code

File Name: src/Components/BookDetails.js

JavaScript

```
import React from 'react';
```

```
function BookItem({ title, author, year }) {
  return (
    <div style={{ padding: '12px', margin: '10px 0', borderLeft: '4px solid #007bff',
backgroundColor: '#f8f9fa' }}>
      <h4 style={{ margin: '0 0 5px 0' }}>{title}</h4>
      <p style={{ margin: 0, fontSize: '0.9rem', color: '#555' }}>By {author} |
Published: {year}</p>
    </div>
  );
}
```

```
function BookDetails() {
  const books = [
    { id: 'b1', title: "Eloquent JavaScript", author: "Marijn Haverbeke", year: 2018
},
    { id: 'b2', title: "Designing Data-Intensive Applications", author: "Martin
Kleppmann", year: 2017 },
    { id: 'b3', title: "You Don't Know JS Yet", author: "Kyle Simpson", year: 2020 }
  ];
```

```
  return (
    <div>
      <h3> Book Resource Library</h3>
      <p style={{ color: '#666' }}>Rendering extracted child items using map()
loops with unique identities.</p>
```

```
    { /* React Map execution: Transforming array data into JSX collection
elements */}
    {books.map((book) => (
      <BookItem
        key={book.id}
        title={book.title}
        author={book.author}
        year={book.year}
      />
    ))}
  </div>
);
}
```

```
export default BookDetails;
```

File Name: src/Components/BlogDetails.js

JavaScript

```
import React from 'react';
```

```
function BlogDetails() {
  const posts = [
    { id: 'p1', title: "Mastering React State Hooks", category: "Development",
isFeatured: true },
    { id: 'p2', title: "A Guide to Tailwind CSS Layouts", category: "Design",
isFeatured: false },
    { id: 'p3', title: "Understanding AI Core Orchestrators", category: "AI/ML",
isFeatured: true }
  ];

  return (
    <div>
      <h3>  Editorial Blog Catalog</h3>

      {posts.map((post) => (
        <div key={post.id} style={{ padding: '15px', borderBottom: '1px solid #ddd'
}}>
          <h4 style={{ margin: '0 0 5px 0', display: 'flex', alignItems: 'center', gap:
'10px' }}>
            {post.title}

            {/* Short-Circuit Rendering: Tag renders only if isFeatured evaluates to
true */}
            {post.isFeatured && (
              <span style={{ backgroundColor: '#ffc107', color: '#212529', fontSize:
'0.75rem', padding: '2px 6px', borderRadius: '4px' }}>
                Featured Star
              </span>
            )}
          </h4>
          <span style={{ fontSize: '0.8rem', color: '#6c757d', textTransform:
'uppercase' }}>{post.category}</span>
        </div>
      ))}
    </div>
```



```
);  
}
```

export default BlogDetails;

File Name: src/Components/CourseDetails.js

JavaScript

```
import React, { useState } from 'react';  
function MaintenanceBanner({ active }) {  
  if (active) {  
    return null;  
  }  
  return (  
    <div style={{ backgroundColor: '#d4edda', color: '#155724', padding: '10px',  
borderRadius: '4px', margin: '10px 0' }}>  
      ✓ All systems online. Syncing active registrations.  
    </div>  
  );  
}
```

```
function CourseDetails() {  
  const [hideBanner, setHideBanner] = useState(false);  
  const [viewFormat, setViewFormat] = useState('list'); // 'list' or 'grid'  
  const courses = [  
    { id: 'c1', title: "Full-Stack Web Architectures", duration: "12 Weeks" },  
    { id: 'c2', title: "Deep Learning Foundations", duration: "8 Weeks" },  
    { id: 'c3', title: "Advanced Prompt Engineering Matrix", duration: "4 Weeks" }  
  ];  
  return (  
    <div>  
      <h3>🎓 Learning Course Curriculum</h3>  
      <MaintenanceBanner active={hideBanner} />  
  
      <div style={{ marginBottom: '15px', display: 'flex', gap: '10px' }}>  
        <button onClick={() => setHideBanner(!hideBanner)} style={{ fontSize:  
'0.8rem', padding: '5px 10px' }}>  
          Toggle Status Notification Banner  
        </button>  
        <button onClick={() => setViewFormat(viewFormat === 'list' ? 'grid' : 'list')}  
style={{ fontSize: '0.8rem', padding: '5px 10px' }}>  
          Switch View Mode
```

```

        </button>
      </div>
      {viewFormat === 'list' ? (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '8px' }}>
          {courses.map(course => (
            <div key={course.id} style={{ border: '1px solid #ccc', padding: '10px',
borderRadius: '4px' }}>
              <strong>{course.title}</strong> — {course.duration}
            </div>
          ))}
        </div>
      ) : (
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '12px' }}>
          {courses.map(course => (
            <div key={course.id} style={{ border: '2px dashed #007bff', padding:
'15px', borderRadius: '6px' }}>
              <h4>{course.title}</h4>
              <p style={{ margin: 0, color: '#007bff' }}>{course.duration}</p>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

```

export default CourseDetails;

File Name: src/Components/BloggerDashboard.js

JavaScript

```

import React, { useState } from 'react';
import BookDetails from './BookDetails';
import BlogDetails from './BlogDetails';
import CourseDetails from './CourseDetails';
function BloggerDashboard() {
  const [activeTab, setActiveTab] = useState('books');
  let displayMarkupContent;
  if (activeTab === 'books') {
    displayMarkupContent = <BookDetails />;
  } else if (activeTab === 'blogs') {
    displayMarkupContent = <BlogDetails />;
  }
}

```

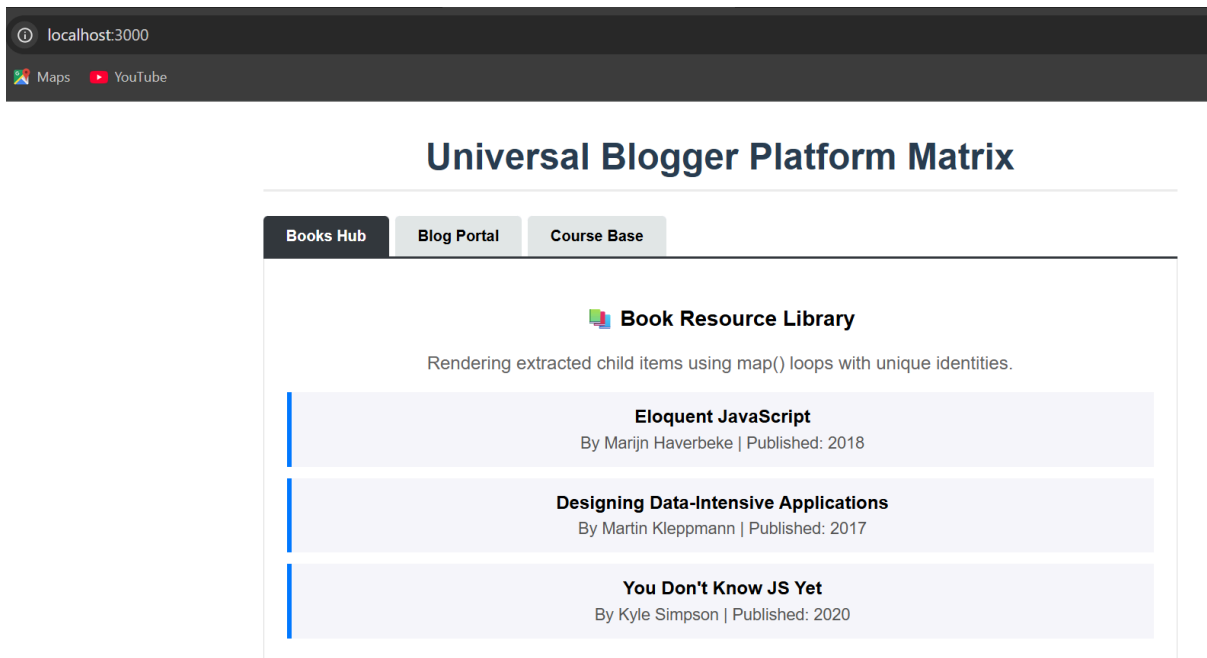
```

    } else if (activeTab === 'courses') {
      displayMarkupContent = <CourseDetails />;
    } else {
      displayMarkupContent = <p>No items found.</p>;
    }
  }
  const getTabStyle = (tabName) => ({
    padding: '10px 20px',
    cursor: 'pointer',
    border: 'none',
    backgroundColor: activeTab === tabName ? '#343a40' : '#e2e6ea',
    color: activeTab === tabName ? 'white' : 'black',
    borderRadius: '4px 4px 0 0',
    fontWeight: 'bold'
  });
  return (
    <div style={{ fontFamily: 'Arial, sans-serif', maxWidth: '800px', margin: '30px auto', padding: '0 20px' }}>
      <h1 style={{ textAlign: 'center', color: '#2c3e50', borderBottom: '2px solid #eee', paddingBottom: '10px' }}>
        Universal Blogger Platform Matrix
      </h1>
      <div style={{ display: 'flex', gap: '5px', borderBottom: '2px solid #343a40', marginTop: '20px' }}>
        <button onClick={() => setActiveTab('books')}
          style={getTabStyle('books')}>Books Hub</button>
        <button onClick={() => setActiveTab('blogs')}
          style={getTabStyle('blogs')}>Blog Portal</button>
        <button onClick={() => setActiveTab('courses')}
          style={getTabStyle('courses')}>Course Base</button>
      </div>
      <div style={{ padding: '20px', border: '1px solid #ddd', borderTop: 'none', borderRadius: '0 0 4px 4px', minHeight: '300px' }}>
        {displayMarkupContent}
      </div>
    </div>
  );
}
export default BloggerDashboard;
File Name: src/App.js
import React from 'react';

```

```
import './App.css';
import BloggerDashboard from './Components/BloggerDashboard';
function App() {
  return (
    <div className="App">
      <BloggerDashboard />
    </div>
  );
}
export default App;
```

OUTPUT:



Universal Blogger Platform Matrix

Books Hub

Blog Portal

Course Base

 Editorial Blog Catalog

Mastering React State Hooks

Featured Star

DEVELOPMENT

A Guide to Tailwind CSS Layouts

DESIGN

Understanding AI Core Orchestrators

Featured Star

AI/ML

Universal Blogger Platform Matrix

Books Hub

Blog Portal

Course Base

 Learning Course Curriculum

✓ All systems online. Syncing active registrations.

Toggle Status Notification Banner

Switch View Mode

Full-Stack Web Architectures

12 Weeks

Deep Learning Foundations

8 Weeks

Advanced Prompt Engineering Matrix

4 Weeks